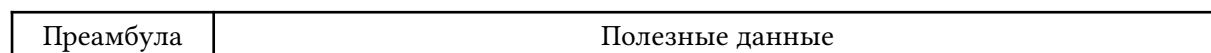


NRFM: Спецификация пакетов для передачи данных по радиоканалу

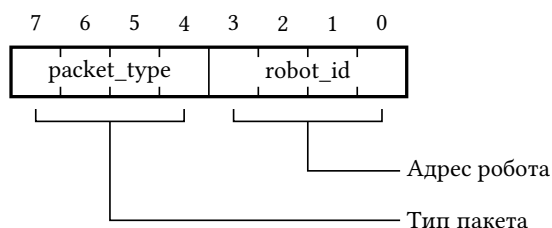
v0.2.0

SPbUnited

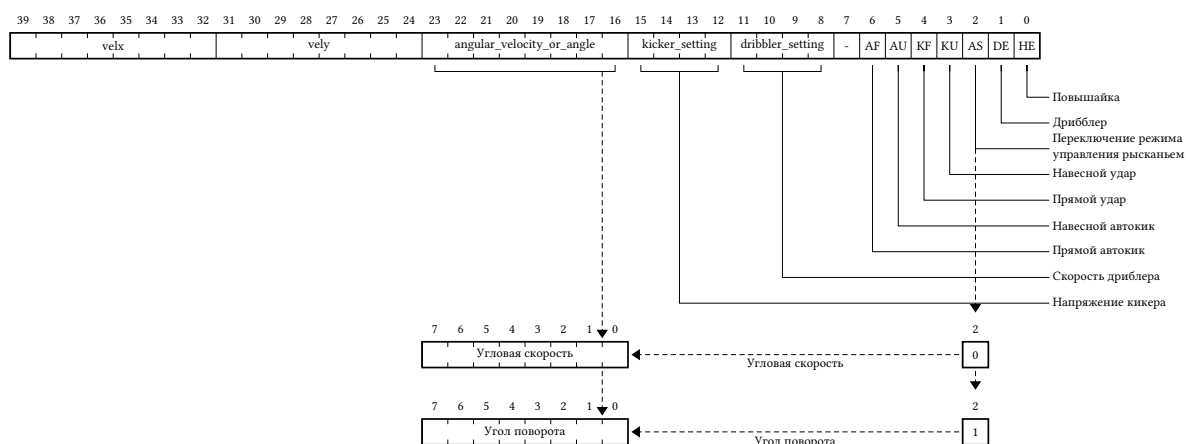
Общий формат пакета представлен ниже. Первый байт - преамбула, определяющая тип пакета. После идут полезные данные в соответствии со спецификацией.



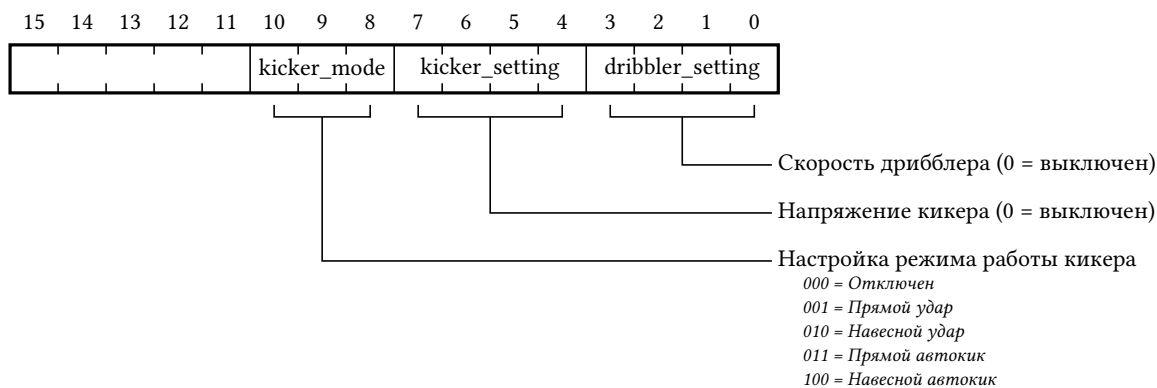
Преамбула



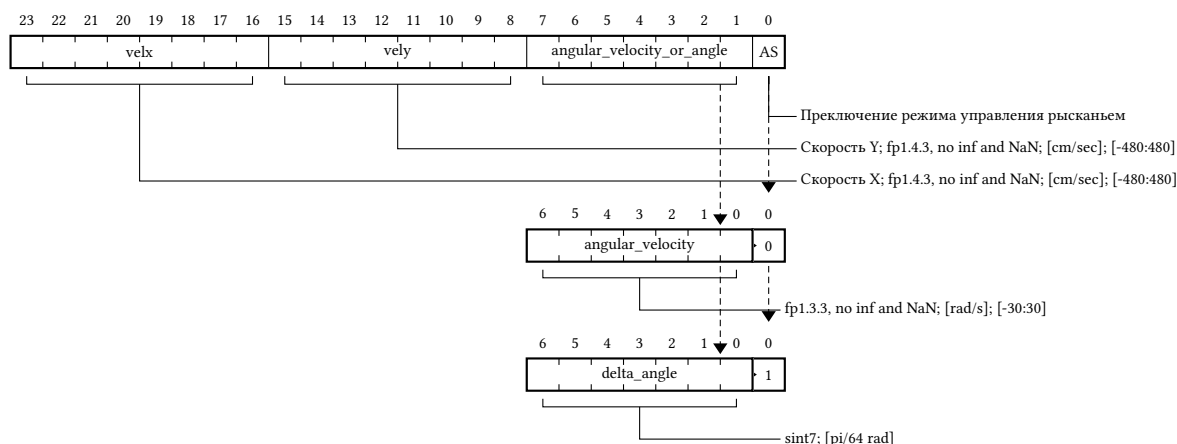
0. [0x0: old_format] Пакет старого формата



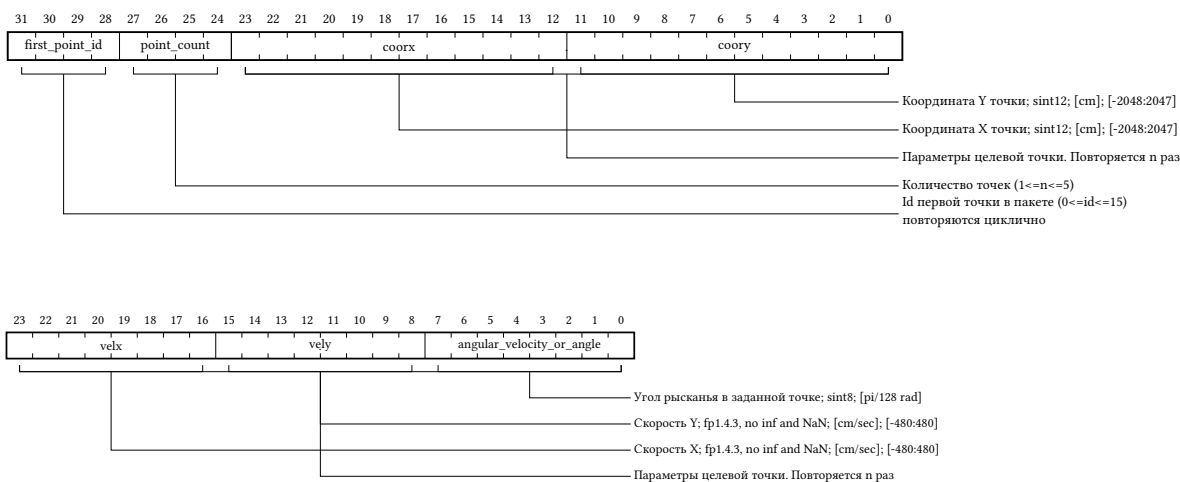
1. [0x1: kicker_and_dribbler] Настройка параметров кикера и дриблера



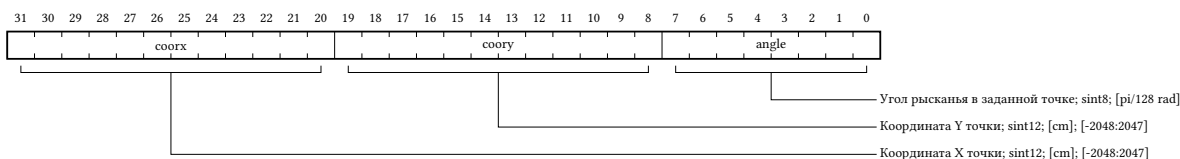
2. [0x2: speed_control] Выдача задания по скорости



3. [0x3: coordinate_control] Управление по координатам



4. [0x4: global_coordinate] Сообщение с текущими координатами робота



5. [0x5: cap_vel_and_accel] Ограничение максимальной скорости и ускорения



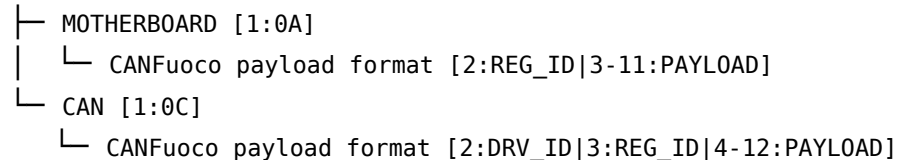
10. [0xA: debug_override] Служебные сообщения настройки

Для передачи служебных сообщений регламентируется ряд форматов. Все они используют байты полезной нагрузки с 1 по 31, и их описание приведено ниже.

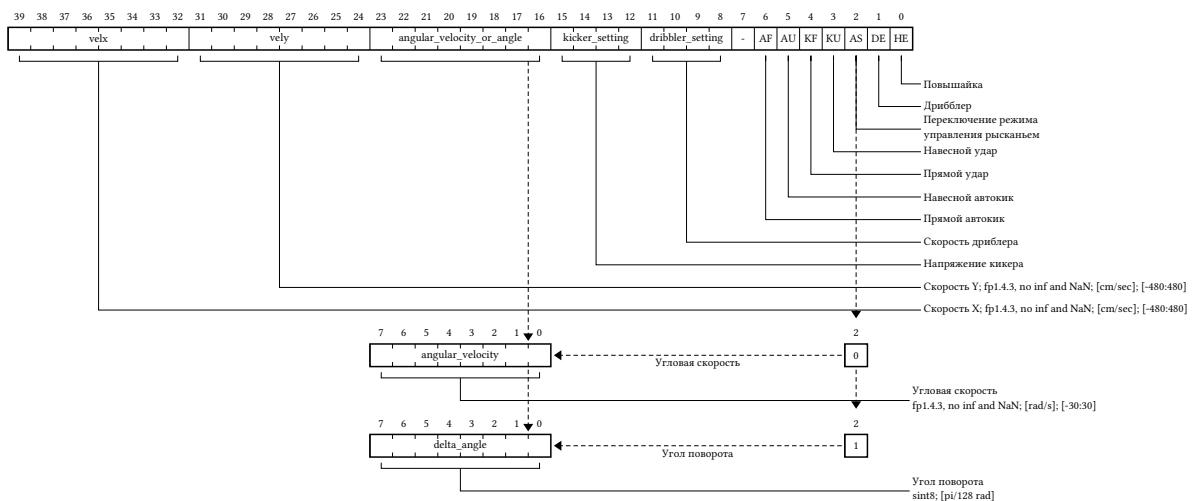
Каждый элемент дерева показывает чему соответствуют байты с указанными номерами. Например:

- [1:0C] говорит, что байт 1 равен значению 0x0C,
- [3:REG_ID|4-6:PAYLOAD] говорит, что байт 3 - соответствует значению REG_ID, байты 4-6 - значению PAYLOAD.

PREAMBLE [0:Ax, A - Packet ID (10), x - ROBOT_ID]

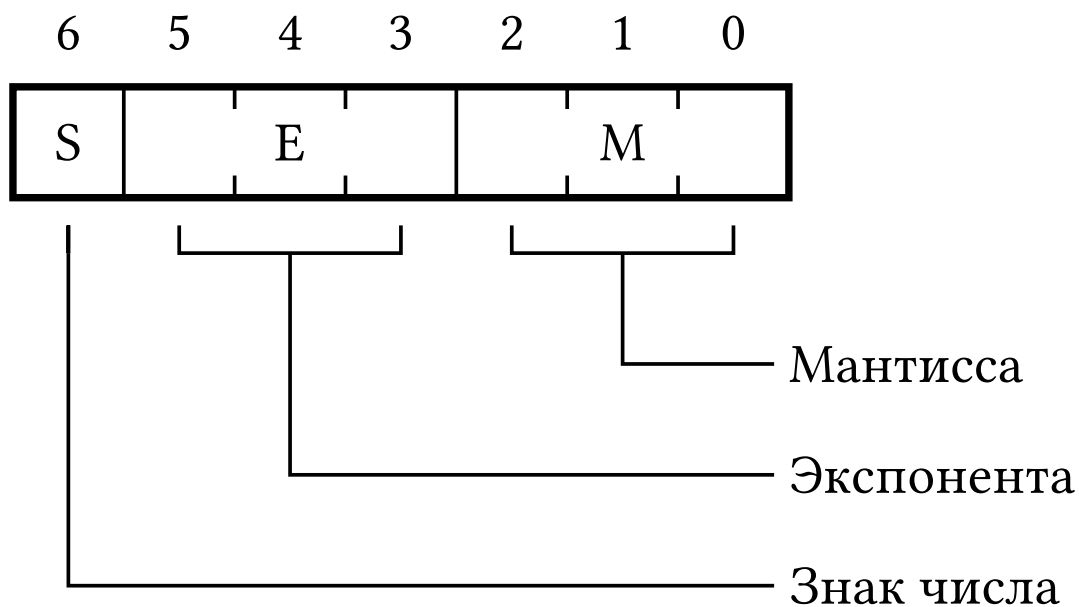


15. [0xF: new_old_format] Модифицированный пакет старого формата

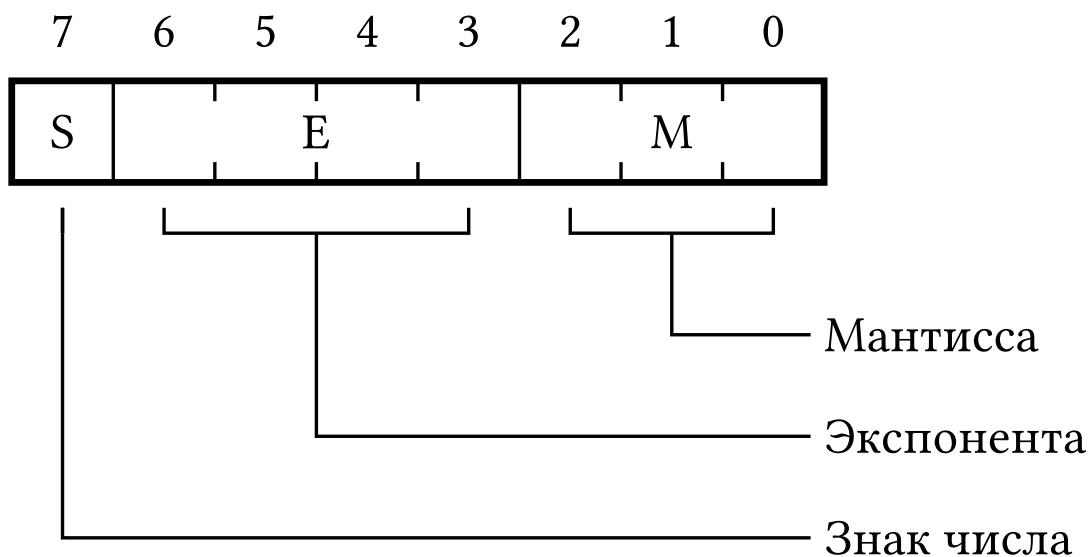


Спецификация чисел с плавающей запятой

FP1.3.3



FP1.4.3



Функции кодирования и декодирования

Python

```
import math
```

```
def float_to_minifloat(x, exponent_bits, mantissa_bits):
```

```

if x == 0.0:
    return (0, 0, 0)
sign = 0 if x > 0 else 1
x = abs(x)
m, e = math.frexp(x)
significand = m * 2 # Now in [1.0, 2.0)
exponent = e - 1
fractional_part = significand - 1.0
bias = (1 << (exponent_bits - 1)) - 1
stored_exponent = exponent + bias
max_exp = (1 << exponent_bits) - 1
max_significand = (1 << mantissa_bits) - 1
# print()
# print("signif\texp\tfrac\tbias\texp\tmax_exp")
# print(
#     significand, exponent, fractional_part, bias, stored_exponent, max_exp,
sep="\t"
# )

is_subnormal = False

if stored_exponent > max_exp:
    return (sign, max_exp, max_significand)
elif stored_exponent < -mantissa_bits:
    return (sign, 0, 0)
else:
    if stored_exponent <= 0:
        fractional_part = (fractional_part + 1) * 2**stored_exponent
        stored_exponent = 0
        is_subnormal = True

    scaled = fractional_part * (2**mantissa_bits)
    rounded_mantissa = round(scaled)
    # print("scaled\trounded")
    # print(scaled, rounded_mantissa, sep="\t")
    if rounded_mantissa >= (1 << mantissa_bits) and not is_subnormal:
        stored_exponent += 1
        mantissa = 0
        if stored_exponent > max_exp:
            return (sign, max_exp, max_significand)
    else:
        mantissa = rounded_mantissa
    # if stored_exponent == 0:
    #     mantissa +=
    return (sign, stored_exponent, mantissa)

def minifloat_to_float(sign, stored_exponent, mantissa, exponent_bits,
mantissa_bits):
    bias = (1 << (exponent_bits - 1)) - 1
    exponent2 = 2 ** (stored_exponent - bias - mantissa_bits)
    normalizer = (stored_exponent != 0) * (2 ** (stored_exponent - bias))
    print(bias, exponent2, mantissa, normalizer)
    return (-1) ** sign * (exponent2 * mantissa + normalizer)

```

```

if __name__ == "__main__":

    def minifloat_to_binary(
        sign, stored_exponent, mantissa, exponent_bits, mantissa_bits
    ):
        exponent_str = format(stored_exponent, f"0{exponent_bits}b")
        mantissa_str = format(mantissa, f"0{mantissa_bits}b")
        return (
            f"{sign} {stored_exponent} {mantissa}\t{sign}{exponent_str}"
            f"{mantissa_str}"
        )

    # Example usage for 1.4.3 format (1 sign, 4 exponent, 3 mantissa bits)
    x = 80
    sign, exp, mantissa = float_to_minifloat(x, 4, 3)
    binary_1_4_3 = minifloat_to_binary(sign, exp, mantissa, 4, 3)
    print(
        "1.4.3 format:", binary_1_4_3, minifloat_to_float(sign, exp, mantissa, 4, 3)
    ) # Output: 01000110 (7 bits, but may need adjustment)

    # Example usage for 1.3.4 format (1 sign, 3 exponent, 4 mantissa bits)
    sign, exp, mantissa = float_to_minifloat(x, 3, 4)
    binary_1_3_4 = minifloat_to_binary(sign, exp, mantissa, 3, 4)
    print(
        "1.3.4 format:", binary_1_3_4, minifloat_to_float(sign, exp, mantissa, 3, 4)
    ) # Output: 01001100

    # Example usage for 1.3.3 format (1 sign, 3 exponent, 3 mantissa bits)
    sign, exp, mantissa = float_to_minifloat(x, 3, 3)
    binary_1_3_3 = minifloat_to_binary(sign, exp, mantissa, 3, 3)
    print(
        "1.3.3 format:", binary_1_3_3, minifloat_to_float(sign, exp, mantissa, 3, 3)
    ) # Output: 0100110 (7 bits, but may need adjustment)

```